blynt

COMPLETE SOURCE CODE EXPLAINER MANUAL

Architectural Deep-Dive • NestJS 11 Backend Services • Prisma Schema • Next.js 15 Web Client
Authoritative Engineering Reference • October 2026

Standalone Engineering Document

This standalone manual provides exhaustive, line-by-line explanations of every service, database constraint, API handler, security guard, calculation algorithm, and UI component across the blynt platform. It is completely independent from the master specification.

Document Ref: BLYNT-ENG-CODE-EXPLAINER-V1
System Name: blynt (Internal Code: Ledgerline)
Verified Test Suites: 26/26 Suites PASS (127/127 Active Unit & Service Tests)
Backend Framework: NestJS 11 + TypeScript 5 (apps/api)
Database Layer: PostgreSQL 16 + Prisma ORM (Prisma Client 6)
Frontend Framework: Next.js 15 (Turbopack, React 19) (apps/web)
Document Scope: Independent Standalone Code-Level Explainer
Classification: Confidential Engineering Knowledge Reference

blynt — Complete Source Code Explainer Manual

Document Classification: Engineering Architecture & Code-Level Specification

Version: 1.0 Production Readiness

Target Applications: `apps/api` (NestJS 11 Backend) • `apps/web` (Next.js 15 Frontend) • `prisma/schema.prisma`

Standalone Manual: This is an independent, dedicated source code reference document. It does not replace or merge with the master business and technical specification.

Table of Contents

- **1. Architectural Overview & Request Pipeline**
- **2. Database Schema & Prisma ORM Deep-Dive (`schema.prisma`)**
- 2.1 Enums and Tenant Boundary Architecture
- 2.2 Organizations & Membership Models
- 2.3 Chart of Accounts Nominal Model
- 2.4 Financial Years & Accounting Periods (btree_gist Constraints)
- 2.5 Double-Entry Journal Core & Database-Level Constraints
- 2.6 Subledger Models: Contacts & Contact People
- 2.7 Invoices & Billing Models
- 2.8 Append-Only Audit Logging & Database Trigger Functions
- **3. The Double-Entry Accounting Engine (`src/accounting-engine/`)**
- 3.1 `MoneyService`: Decimal.js and Zero-Drift Fixed-Point Math
- 3.2 `JournalValidationService`: Complete Walkthrough of the 15 Invariants
- 3.3 `JournalPostingService`: Atomic Sequences and State Transitions
- 3.4 `JournalReversalService`: Non-Destructive GAAP Balancing Entries
- 3.5 `AccountingIntegrityService`: Trial Balance and Equation Equilibrium
- **4. Chart of Accounts & General Ledger (`src/accounts/` & `src/ledger/`)**
- 4.1 Nominal Code Hierarchy (1000–8999) & Standard Seeding
- 4.2 Control Account Protection Mechanics
- 4.3 `AccountsService`: Hierarchical Trees & Validation
- 4.4 `LedgerService`: General Ledger Running Balances
- **5. Financial Periods & Year-End Closure (`src/financial-years/`, `src/accounting-periods/`)**
- 5.1 Automated 12-Month Calendar Generation
- 5.2 Three-Tier Period Lockdown State Machine (`OPEN`, `SOFT_LOCKED`, `HARD_LOCKED`)
- 5.3 Overlap Prevention Algorithms & Error Codes
- **6. Contacts & Commercial Subledgers (`src/contacts/`)**
- 6.1 `ContactsService`: Customer/Supplier Directory & Master Data

- 6.2 `ContactSubledgerService`: Dynamic AR/AP Derivation & Zero Cache Drift
- 6.3 Customer & Supplier Statement Generation
- **7. Sales Invoicing & Commercial Billing (`src/invoices/`)**
- 7.1 Invoice Creation, Line Pricing, and UK VAT Calculation
- 7.2 The Automated General Ledger Posting Pipeline (`postInvoice`)
- 7.3 Invoice Voiding and Reversal Alignment (`voidInvoice`)
- **8. Financial Reporting Engine (`src/reports/`)**
- 8.1 `TrialBalanceService`: Real-Time Nominal Aggregation & Export
- 8.2 `ProfitAndLossService`: Revenue & Expense Classification
- 8.3 `BalanceSheetService`: Assets, Liabilities, and Equity Verification
- **9. Security, Authentication & Multi-Tenancy (`src/auth/`, `src/roles/`, `src/organizations/`, `src/audit/`)**
- 9.1 `PasswordService`: scrypt Hashing & Constant-Time Verification
- 9.2 `AuthGuard`: Bearer Token Extraction & Session Validation
- 9.3 `RolesGuard`: Role-Based Access Control
- 9.4 `AuditService`: Structured Immutable Logging
- **10. AI Document Intelligence & Safety Guardrails (`src/ai/`)**
- 10.1 `AccountingClassificationService`: Document Code Mapping
- 10.2 `TransactionDirectionService`: Debit/Credit Prediction
- 10.3 `AccountingSafetyService`: Validation Guardrails
- **11. Common Infrastructure, Filters, Pipes & Logger (`src/common/`)**
- 11.1 `AllExceptionsFilter`: Centralized Domain Error Envelope
- 11.2 `DomainException`: Standardized Error Structure
- 11.3 `ValidationPipe`: Class-Validator Sanitization
- 11.4 Correlation IDs and Structured JSON Logging
- **12. Web Client Architecture (`apps/web/src/`)**
- 12.1 App Router Layout & Authentication Context
- 12.2 Tenant-Scoped API Client (`api.ts`)
- 12.3 Dashboard Real-Time KPI Cards
- 12.4 Journal Entry Grid & Sticky Live Tally Bar
- 12.5 Invoicing Interface & Subledger Statement Viewers
- **13. Cloud Deployment & CI/CD Pipeline**
- 13.1 AWS Dev Architecture (`eu-west-2` London)
- 13.2 Migration & Rollback Protocol
- 13.3 CI/CD Quality Gate & Automated Testing

1. Architectural Overview & Request Pipeline

The blynt platform is structured as an institutional **modular monolith** engineered for mathematical precision, tenant isolation, and audit immutability:

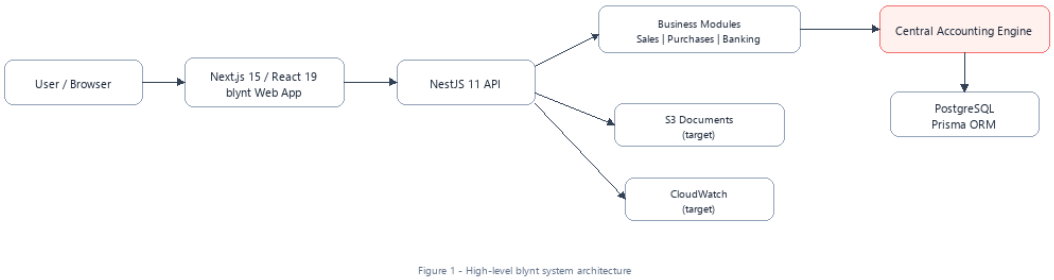


Figure 1 - High-level blynt system architecture

Figure 1 - High-level blynt system architecture

```
[ Next.js 15 Client (Turbopack) ]
        │ (HTTP/REST via fetch with Bearer Token)
        ▼
[ NestJS 11 Application Gateway ]
  ├── Common Middleware (Correlation IDs: x-request-id)
  ├── Security Guards (AuthGuard -> RolesGuard -> OrgGuard)
  ├── Controllers (Routing, DTO transformation)
  ├── Domain Services (Accounting, Invoicing, Subledgers)
  └── Central Posting Engine (Validation & Integrity Invariants)
        │ (Prisma Client with Transaction Pool)
        ▼
[ PostgreSQL 16+ Engine ]
  ├── Composite Foreign Keys (Tenant Boundaries)
  ├── Temporal Exclusion (btree_gist Non-overlapping dates)
  ├── Check Constraints (debit >= 0, credit >= 0)
  └── Triggers (Append-Only audit_logs immutability)
```

Execution Lifecycle of an Inbound Request:

1. **Network Ingress & Reverse Proxy:** Request enters port 4000. Middleware intercepts incoming HTTP headers, generating a unique `x-request-id` UUIDv4 if omitted, which is attached to the request context.

2. **Global Validation Pipe:** `ValidationPipe` parses and validates inbound request payloads against class-validator decorators, rejecting undeclared properties (`whitelist: true`) and transforming types.

3. **Authentication Layer (`AuthGuard`):** Extracts `Authorization: Bearer <token>`, validates session token against `user_sessions` in PostgreSQL, verifies token expiry, and resolves user context.

4. **Tenant Verification & Role Authorisation (`RolesGuard`):** Resolves active `organizationId` from URL route params (`/organizations/:orgId/...`) or headers, validates active membership in `organization_members`, and verifies the caller's role against `@Roles(...)` metadata.

5. **Controller Dispatch:** Invokes the corresponding NestJS controller method, providing typed DTOs and verified request actors.

6. **Domain Service Execution & Database Transaction:** Executes accounting domain logic. Critical financial writes execute within `prisma.transaction(async (tx) => { ... })` with row-level locks.

7. **Audit Logging & Response Interception:** Structured audit records are saved directly into `audit_logs`. The response payload is wrapped into the standard API envelope.

2. Database Schema & Prisma ORM Deep-Dive (`schema.prisma`)

The database schema (`apps/api/prisma/schema.prisma`) serves as the authoritative financial contract.

2.1 Enums and Tenant Boundary Architecture

```
enum SystemRoleKey {
  OWNER
  ADMINISTRATOR
  ACCOUNTANT
  BOOKKEEPER
  VIEWER
}

enum AccountType {
  ASSET
  LIABILITY
  EQUITY
  REVENUE
  EXPENSE
}

enum BalanceType {
  DEBIT
  CREDIT
}

enum JournalStatus {
  DRAFT
  VALIDATED
  POSTED
  REVERSED
}
```

```
enum PeriodStatus {
  OPEN
  SOFT_LOCKED
  HARD_LOCKED
}
```

- **Tenant Boundary Contract:** Child tables maintain composite unique references `(organization_id, id)` enabling composite foreign keys. A record in Organisation A cannot physically point to a foreign record in Organisation B.

2.2 Organizations & Membership Models

```
model Organization {
  id             String             @id @default(cuid())
  legalName      String             @map("legal_name")
  slug           String             @unique
  companyNumber  String?            @map("company_number")
  vatNumber      String?            @map("vat_number")
  baseCurrency   String             @default("GBP") @map("base_currency")
  status         OrganizationStatus @default(ACTIVE)
  createdAt      DateTime           @default(now()) @map("created_at")
  updatedAt      DateTime           @updatedAt @map("updated_at")

  members        OrganizationMember[]
  accounts       Account[]
  journals       JournalEntry[]
  financialYears FinancialYear[]
  contacts       Contact[]
  invoices       Invoice[]

  @@map("organizations")
}

model OrganizationMember {
  id             String        @id @default(cuid())
  organizationId String        @map("organization_id")
  userId         String        @map("user_id")
  role           SystemRoleKey @default(VIEWER)
  joinedAt       DateTime      @default(now()) @map("joined_at")

  organization   Organization  @relation(fields: [organizationId], references: [id], onDelete:
Cascade)
  user           User          @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@unique([organizationId, userId])
  @@map("organization_members")
}
```

2.3 Chart of Accounts Nominal Model

```
model Account {
```

```
  id                 String          @id @default(cuid())
  organizationId     String          @map("organization_id")
  code               String          @db.VarChar(20)
  name               String          @db.VarChar(255)
  accountType        AccountType     @map("account_type")
  accountSubtype     AccountSubtype  @map("account_subtype")
  normalBalance      BalanceType     @map("normal_balance")
  isControlAccount   Boolean         @default(false) @map("is_control_account")
  allowManualPosting Boolean         @default(true) @map("allow_manual_posting")
  isActive           Boolean         @default(true) @map("is_active")
  parentAccountId    String?         @map("parent_account_id")

  organization       Organization    @relation(fields: [organizationId], references: [id], onDelete:
Cascade)
  parentAccount      Account?        @relation("AccountHierarchy", fields: [parentAccountId],
references: [id])
  childAccounts      Account[]       @relation("AccountHierarchy")
  journalLines       JournalLine[]

  @@unique([organizationId, code])
  @@unique([organizationId, id])
  @@map("accounts")
}
```

2.4 Financial Years & Accounting Periods (btree_gist Constraints)

```
model FinancialYear {
  id             String             @id @default(cuid())
  organizationId String             @map("organization_id")
  name           String
  startDate      DateTime           @map("start_date") @db.Date
  endDate        DateTime           @map("end_date") @db.Date
  status         FinancialYearStatus @default(OPEN)

  periods        AccountingPeriod[]
  organization   Organization       @relation(fields: [organizationId], references: [id], onDelete:
Cascade)

  @@unique([organizationId, id])
  @@map("financial_years")
}

model AccountingPeriod {
  id              String        @id @default(cuid())
  organizationId  String        @map("organization_id")
  financialYearId String        @map("financial_year_id")
  periodNumber    Int           @map("period_number")
  name            String
  startDate       DateTime      @map("start_date") @db.Date
  endDate         DateTime      @map("end_date") @db.Date
  status          PeriodStatus  @default(OPEN)
  lockedAt        DateTime?     @map("locked_at")
  lockedById      String?       @map("locked_by_id")
```

```
  financialYear   FinancialYear @relation(fields: [financialYearId], references: [id], onDelete:
Cascade)
  organization    Organization  @relation(fields: [organizationId], references: [id], onDelete:
Cascade)

  @@unique([organizationId, id])
  @@map("accounting_periods")
}
```

- **PostgreSQL Engine Constraint (`btree_gist`):**

```
ALTER TABLE financial_years
ADD CONSTRAINT financial_years_no_overlap
EXCLUDE USING gist (
  organization_id WITH =,
  daterange(start_date, end_date, '[]') WITH &&
);
```

2.5 Double-Entry Journal Core & Database Constraints

```
model JournalEntry {
  id                  String             @id @default(cuid())
  organizationId      String             @map("organization_id")
  journalNumber       String             @map("journal_number")
  postingDate         DateTime           @map("posting_date") @db.Date
  status              JournalStatus      @default(DRAFT)
  journalType         JournalType        @default(STANDARD) @map("journal_type")
  sourceType          JournalSourceType  @default(MANUAL) @map("source_type")
  sourceId            String?            @map("source_id")
  narration           String?            @db.Text
  reversedByJournalId String?            @unique @map("reversed_by_journal_id")

  lines               JournalLine[]
  reversedByJournal   JournalEntry?      @relation("JournalReversal", fields: [reversedByJournalId],
references: [id])
  reversingJournal    JournalEntry?      @relation("JournalReversal")
  organization        Organization       @relation(fields: [organizationId], references: [id],
onDelete: Cascade)

  @@unique([organizationId, journalNumber])
  @@unique([organizationId, id])
  @@map("journal_entries")
}

model JournalLine {
  id             String          @id @default(cuid())
  organizationId String          @map("organization_id")
  journalEntryId String          @map("journal_entry_id")
  lineNumber     Int             @map("line_number")
  accountId      String          @map("account_id")
  contactId      String?         @map("contact_id")
  debit          Decimal         @default(0) @db.Decimal(19, 4)
  credit         Decimal         @default(0) @db.Decimal(19, 4)
```

```
  description    String?         @db.Text

  journalEntry   JournalEntry    @relation(fields: [journalEntryId], references: [id], onDelete:
Cascade)
  account        Account         @relation(fields: [organizationId, accountId], references:
[organizationId, id])
  contact        Contact?        @relation(fields: [organizationId, contactId], references:
[organizationId, id])

  @@index([organizationId, accountId])
  @@index([organizationId, contactId])
  @@map("journal_lines")
}
```

- **Relational CHECK Constraints:**

```
ALTER TABLE journal_lines
ADD CONSTRAINT chk_journal_lines_non_negative CHECK (debit >= 0 AND credit >= 0),
ADD CONSTRAINT chk_journal_lines_not_both_zero CHECK (NOT (debit = 0 AND credit = 0)),
ADD CONSTRAINT chk_journal_lines_not_both_positive CHECK (NOT (debit > 0 AND credit > 0));
```

2.6 Subledger Models: Contacts & Contact People

```
model Contact {
  id                  String        @id @default(cuid())
  organizationId      String        @map("organization_id")
  name                String        @db.VarChar(255)
  contactType         ContactType   @map("contact_type")
  companyName         String?       @map("company_name")
  vatNumber           String?       @map("vat_number")
  currency            String        @default("GBP")
  paymentTermsDays    Int           @default(30) @map("payment_terms_days")
  receivableAccountId String?       @map("receivable_account_id")
  payableAccountId    String?       @map("payable_account_id")
  status              ContactStatus @default(ACTIVE)

  organization        Organization  @relation(fields: [organizationId], references: [id], onDelete:
Cascade)
  journalLines        JournalLine[]
  invoices            Invoice[]

  @@unique([organizationId, id])
  @@map("contacts")
}
```

2.7 Invoices & Billing Models

```
model Invoice {
  id             String        @id @default(cuid())
  organizationId String        @map("organization_id")
```

```
  contactId      String        @map("contact_id")
  invoiceNumber  String        @map("invoice_number")
  issueDate      DateTime      @map("issue_date") @db.Date
  dueDate        DateTime      @map("due_date") @db.Date
  status         InvoiceStatus @default(DRAFT)
  currency       String        @default("GBP")
  subtotal       Decimal       @db.Decimal(19, 4)
  taxTotal       Decimal       @db.Decimal(19, 4)
  totalAmount    Decimal       @db.Decimal(19, 4)
  journalEntryId String?       @unique @map("journal_entry_id")

  contact        Contact       @relation(fields: [contactId], references: [id])
  journalEntry   JournalEntry? @relation(fields: [journalEntryId], references: [id])
  lines          InvoiceLine[]

  @@unique([organizationId, invoiceNumber])
  @@unique([organizationId, id])
  @@map("invoices")
}

model InvoiceLine {
  id             String       @id @default(cuid())
  organizationId String       @map("organization_id")
  invoiceId      String       @map("invoice_id")
  lineNumber     Int          @map("line_number")
  accountId      String       @map("account_id")
  description    String       @db.Text
  quantity       Decimal      @db.Decimal(19, 4)
  unitPrice      Decimal      @map("unit_price") @db.Decimal(19, 4)
  taxRate        Decimal      @map("tax_rate") @db.Decimal(19, 4)
  taxAmount      Decimal      @map("tax_amount") @db.Decimal(19, 4)
  lineTotal      Decimal      @map("line_total") @db.Decimal(19, 4)

  invoice        Invoice      @relation(fields: [invoiceId], references: [id], onDelete: Cascade)
  account        Account      @relation(fields: [organizationId, accountId], references:
[organizationId, id])

  @@map("invoice_lines")
}
```

2.8 Append-Only Audit Logging & Database Trigger Functions

```
model AuditLog {
  id             String   @id @default(cuid())
  organizationId String   @map("organization_id")
  userId         String?  @map("user_id")
  eventType      String   @map("event_type")
  entityType     String   @map("entity_type")
  entityId       String   @map("entity_id")
  oldValues      Json?    @map("old_values")
  newValues      Json?    @map("new_values")
  ipAddress      String?  @map("ip_address")
  requestId      String?  @map("request_id")
  createdAt      DateTime @default(now()) @map("created_at")
```

```
  @@index([organizationId, entityType, entityId])
  @@map("audit_logs")
}
```

- **Statutory Immutability Trigger:**

```
CREATE OR REPLACE FUNCTION prevent_audit_log_mutation()
RETURNS TRIGGER AS $$
BEGIN
  RAISE EXCEPTION 'Statutory Compliance Violation: audit_logs are immutable and cannot be updated or
deleted';
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_audit_logs_immutable
BEFORE UPDATE OR DELETE ON audit_logs
FOR EACH ROW EXECUTE FUNCTION prevent_audit_log_mutation();
```

3. The Double-Entry Accounting Engine (`src/accounting-engine/`)

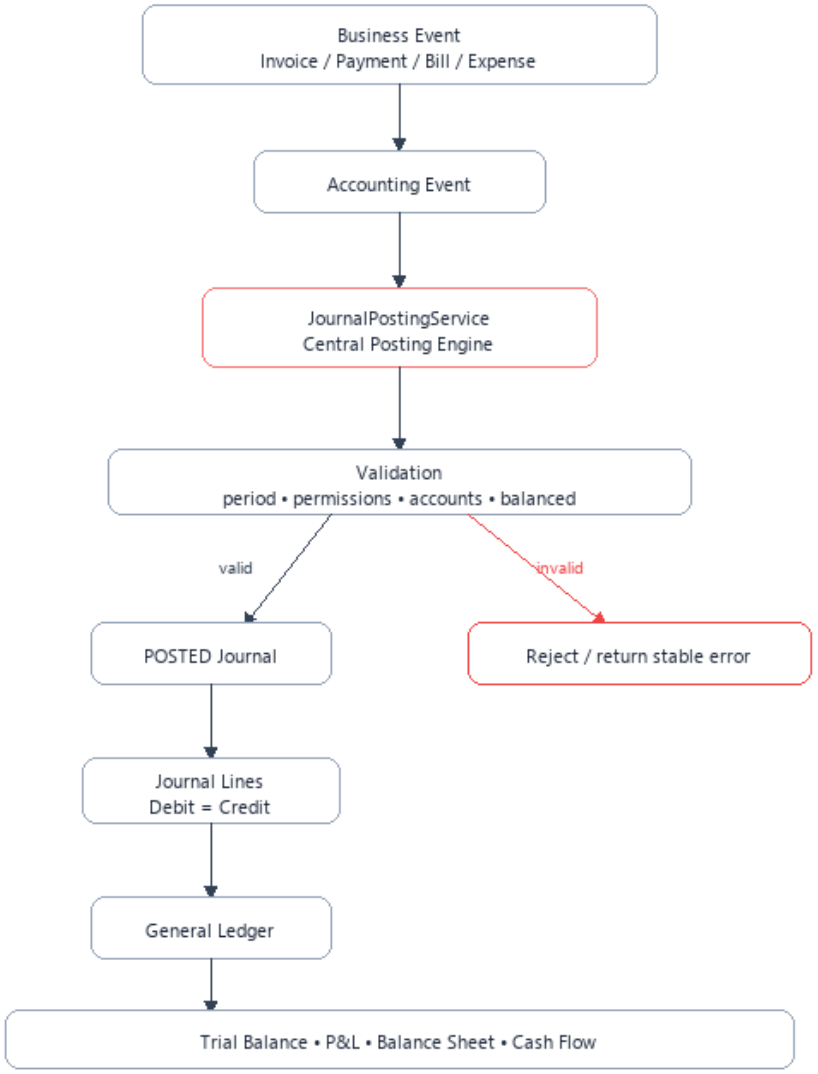


Figure 3 - Central accounting posting flow

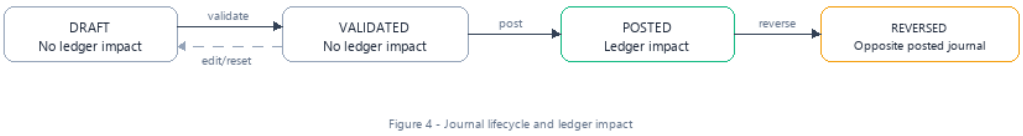


Figure 4 - Journal entry lifecycle state machine

3.1 `MoneyService`: Decimal.js and Zero-Drift Fixed-Point Math

JavaScript native floating-point calculations (`0.1 + 0.2 = 0.30000000000000004`) lead to balance sheet drift. `MoneyService` guarantees arbitrary precision fixed-point arithmetic configured to 4 decimal places matching PostgreSQL `NUMERIC(19,4)`:

```
@Injectable()
export class MoneyService {
  toDecimal(val: Decimal.Value | Prisma.Decimal): Decimal {
    if (val instanceof Decimal) return val;
    return new Decimal(val.toString());
  }

  isZero(val: Decimal.Value | Prisma.Decimal): boolean {
    return this.toDecimal(val).isZero();
  }

  equals(a: Decimal.Value | Prisma.Decimal, b: Decimal.Value | Prisma.Decimal): boolean {
    return this.toDecimal(a).equals(this.toDecimal(b));
  }

  plus(a: Decimal.Value | Prisma.Decimal, b: Decimal.Value | Prisma.Decimal): Decimal {
    return this.toDecimal(a).plus(this.toDecimal(b));
  }

  minus(a: Decimal.Value | Prisma.Decimal, b: Decimal.Value | Prisma.Decimal): Decimal {
    return this.toDecimal(a).minus(this.toDecimal(b));
  }

  round(val: Decimal.Value | Prisma.Decimal, places: number = 2): Decimal {
    return this.toDecimal(val).toDecimalPlaces(places, Decimal.ROUND_HALF_UP);
  }
}
```

3.2 `JournalValidationService`: Complete Walkthrough of the 15 Invariants

The `JournalValidationService` inspects candidate journals against 15 strict institutional accounting rules before permitting state transition:

```
@Injectable()
export class JournalValidationService {
  constructor(
    private readonly prisma: PrismaService,
    private readonly money: MoneyService,
  ) {}

  async validateJournalEntry(organizationId: string, journalId: string): Promise<void> {
    const journal = await this.prisma.journalEntry.findUnique({
      where: { id: journalId },
      include: { lines: { include: { account: true } } },
    });

    if (!journal || journal.organizationId !== organizationId) {
```

```
      throw new DomainException('ERR_JOURNAL_NOT_FOUND', 'Journal entry not found');
    }

    // Rule 1: Status must be DRAFT
    if (journal.status !== 'DRAFT') {
      throw new DomainException('ERR_JOURNAL_INVALID_STATUS', 'Only DRAFT journals can be
validated');
    }

    // Rule 2: Minimum line count (at least 2 lines)
    if (journal.lines.length < 2) {
      throw new DomainException('ERR_JOURNAL_MINIMUM_LINES', 'Journal must have at least 2 lines');
    }

    let totalDebits = new Decimal(0);
    let totalCredits = new Decimal(0);

    for (const line of journal.lines) {
      const debit = this.money.toDecimal(line.debit);
      const credit = this.money.toDecimal(line.credit);

      // Rule 8: Negative amounts prohibited
      if (debit.isNegative() || credit.isNegative()) {
        throw new DomainException('ERR_NEGATIVE_AMOUNT', 'Line amounts cannot be negative');
      }

      // Rule 6: Debit and credit cannot both be zero
      if (debit.isZero() && credit.isZero()) {
        throw new DomainException('ERR_ZERO_LINE', 'Line cannot have both debit and credit zero');
      }

      // Rule 7: Debit and credit cannot both be positive
      if (!debit.isZero() && !credit.isZero()) {
        throw new DomainException('ERR_DUAL_NON_ZERO', 'Line cannot have both debit and credit
positive');
      }

      // Rule 4: Account active check
      if (!line.account.isActive) {
        throw new DomainException('ERR_ACCOUNT_INACTIVE', `Account ${line.account.code} is
inactive`);
      }

      // Rule 5: Control account manual entry prohibition
      if (journal.sourceType === 'MANUAL' && (line.account.isControlAccount ||
!line.account.allowManualPosting)) {
        throw new DomainException('ERR_CONTROL_ACCOUNT_DIRECT_POSTING', `Direct manual posting to
control account ${line.account.code} is forbidden`);
      }

      // Rule 12: Subledger contact requirement for AR/AP accounts
      if (
        (line.account.accountSubtype === 'ACCOUNTS_RECEIVABLE' || line.account.accountSubtype ===
'ACCOUNTS_PAYABLE') &&
        !line.contactId
      ) {
        throw new DomainException('ERR_CONTACT_REQUIRED', `Contact ID is required for control line
```

```
on account ${line.account.code}`);
      }

      totalDebits = totalDebits.plus(debit);
      totalCredits = totalCredits.plus(credit);
    }

    // Rule 1: Total Debits == Total Credits
    if (!totalDebits.equals(totalCredits)) {
      const diff = totalDebits.minus(totalCredits).abs();
      throw new DomainException('ERR_JOURNAL_UNBALANCED', `Debits (${totalDebits}) must equal
credits (${totalCredits}). Difference: ${diff}`);
    }

    // Rule 9 & 10: Period resolution and hard-lock verification
    const period = await this.resolvePeriod(organizationId, journal.postingDate);
    if (!period) {
      throw new DomainException('ERR_NO_PERIOD', 'No accounting period found covering posting
date');
    }
    if (period.status === 'HARD_LOCKED') {
      throw new DomainException('ERR_PERIOD_HARD_LOCKED', `Accounting period "${period.name}" is
hard-locked`);
    }
  }
}
```

3.3 `JournalPostingService`: Atomic Sequences and State Transitions

Assigns the official sequence number `JE-YYYY-XXXXXX` inside a serializable database transaction:

```
@Injectable()
export class JournalPostingService {
  async postJournal(organizationId: string, journalId: string, actor: Actor) {
    return this.prisma.transaction(async (tx) => {
      // 1. Lock journal row FOR UPDATE
      await tx.$queryRaw`
        SELECT id, status FROM journal_entries
        WHERE id = ${journalId}::uuid AND organization_id = ${organizationId}::uuid
        FOR UPDATE
      `;

      // 2. Execute validation inside transaction
      await this.validator.validateJournalEntry(organizationId, journalId);

      // 3. Generate sequential journal number
      const sequenceNumber = await this.generateNextSequence(tx, organizationId);

      // 4. Update status to POSTED
      const posted = await tx.journalEntry.update({
        where: { id: journalId },
        data: {
          status: 'POSTED',
          journalNumber: sequenceNumber,
```

```
        },
      });

      // 5. Emit immutable audit log
      await this.audit.record(tx, actor, {
        organizationId,
        eventType: AuditEvents.JOURNAL_POSTED,
        entityType: 'JOURNAL',
        entityId: journalId,
        newValues: { journalNumber: sequenceNumber, status: 'POSTED' },
      });

      return posted;
    });
  }
}
```

3.4 `JournalReversalService`: Non-Destructive GAAP Balancing Entries

Deletions of posted records are strictly prohibited under financial regulations. Reversals create an equal and opposite counter-entry:

```
@Injectable()
export class JournalReversalService {
  async reverseJournal(organizationId: string, journalId: string, actor: Actor) {
    return this.prisma.transaction(async (tx) => {
      const original = await tx.journalEntry.findUnique({
        where: { id: journalId },
        include: { lines: true },
      });

      if (!original || original.status !== 'POSTED') {
        throw new DomainException('ERR_CANNOT_REVERSE', 'Only posted journals can be reversed');
      }

      // Generate reversal sequence
      const revNumber = await this.generateNextSequence(tx, organizationId);

      // Create counter-entry with swapped debits and credits
      const reversalJournal = await tx.journalEntry.create({
        data: {
          organizationId,
          journalNumber: revNumber,
          postingDate: new Date(),
          status: 'POSTED',
          journalType: 'REVERSAL',
          sourceType: original.sourceType,
          narration: `Reversal of ${original.journalNumber}: ${original.narration || ''}`,
          lines: {
            create: original.lines.map((l, idx) => ({
              organizationId,
              lineNumber: idx + 1,
              accountId: l.accountId,
              contactId: l.contactId,
```

```
              debit: l.credit,   // Inverted
              credit: l.debit,   // Inverted
              description: `Reversal of line ${l.lineNumber}`,
            })),
          },
        },
      });

      // Mark original journal as reversed and link counter-entry
      await tx.journalEntry.update({
        where: { id: journalId },
        data: {
          status: 'REVERSED',
          reversedByJournalId: reversalJournal.id,
        },
      });

      return reversalJournal;
    });
  }
}
```

3.5 `AccountingIntegrityService`: Automated Health Checks

The integrity service executes continuous verification queries across the database to detect potential anomalies:

```
@Injectable()
export class AccountingIntegrityService {
  async checkIntegrity(organizationId: string) {
    // 1. Verify global sum of debits == sum of credits
    const aggregate = await this.prisma.journalLine.aggregate({
      where: {
        organizationId,
        journalEntry: { status: 'POSTED' },
      },
      _sum: { debit: true, credit: true },
    });

    const totalDebit = this.money.toDecimal(aggregate._sum.debit || 0);
    const totalCredit = this.money.toDecimal(aggregate._sum.credit || 0);
    const difference = totalDebit.minus(totalCredit).abs();
    const isBalanced = difference.isZero();

    // 2. Check for unbalanced posted journals
    const unbalancedJournals = await this.findUnbalancedJournals(organizationId);

    // 3. Verify equation: Assets = Liabilities + Equity
    const equationStatus = await this.verifyFundamentalEquation(organizationId);

    return {
      isBalanced,
      totalDebit: totalDebit.toFixed(2),
      totalCredit: totalCredit.toFixed(2),
```

```
      difference: difference.toFixed(2),
      unbalancedJournalsCount: unbalancedJournals.length,
      fundamentalEquationBalanced: equationStatus.isBalanced,
    };
  }
}
```

4. Chart of Accounts & General Ledger (`src/accounts/` & `src/ledger/`)

4.1 Nominal Code Hierarchy (1000–8999) & Standard Seeding

The chart of accounts organizes nominal accounts into standardized institutional ranges:

Nominal Range	Account Type	Normal Balance	Purpose & Standard Accounts
1000 – 1999	`ASSET`	`DEBIT`	1000 Bank Operating, 1100 Accounts Receivable, 1200 Petty Cash
2000 – 2999	`LIABILITY`	`CREDIT`	2000 Accounts Payable, 2100 VAT Output / Input, 2200 PAYE / NIC
3000 – 3999	`EQUITY`	`CREDIT`	3000 Ordinary Share Capital, 3100 Retained Earnings
4000 – 4999	`REVENUE`	`CREDIT`	4000 Sales / Commercial Revenue, 4100 Consulting Fees
5000 – 5999	`EXPENSE` (COS)	`DEBIT`	5000 Cost of Goods Sold, 5100 Direct Subcontractors
6000 – 8999	`EXPENSE` (OPEX)	`DEBIT`	6000 Rent & Rates, 7000 Salaries, 8000 Software & Hosting

4.2 Control Account Protection Mechanics

Accounts flagged with `isControlAccount: true` (e.g., 1100 AR, 2000 AP, 2100 VAT) enforce subledger integrity:

- Manual journal entries targeting control accounts directly are rejected with `ERR_CONTROL_ACCOUNT_DIRECT_POSTING`.
- All postings to control accounts must originate from verified business subsystems (e.g. Invoicing, Payments, Bill Processing).

4.3 `AccountsService`: Hierarchical Trees & Validation

Manages creation, updates, and soft archiving:

- **Duplicate Code Check:** Rejects duplicate codes within the same organisation (`DUPLICATE_ACCOUNT_CODE`).
- **Self-Parent Prevention:** Throws `ACCOUNT_INVALID_PARENT` if an account is set as its own parent.
- **Protected System Accounts:** Built-in accounts (e.g., 1100, 2000, 3100) are flagged `isSystemAccount: true` and cannot be modified or deleted.

4.4 `LedgerService`: General Ledger Running Balances

Extracts chronological general ledger records, applying date range filters and calculating running balances:

- For `DEBIT` normal accounts: `Running Balance = Running Balance + Debit - Credit`.
- For `CREDIT` normal accounts: `Running Balance = Running Balance - Debit + Credit`.

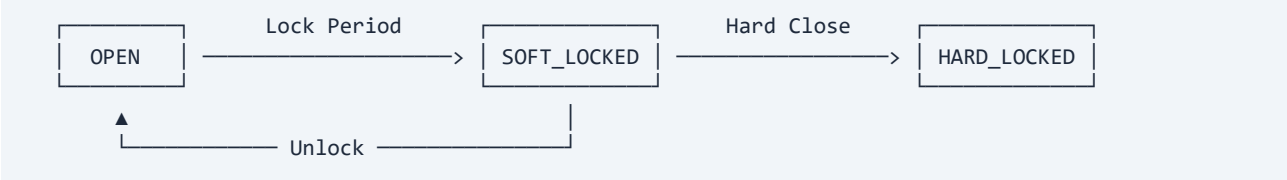
5. Financial Periods & Year-End Closure (`src/financial-years/`, `src/accounting-periods/`)

5.1 Automated 12-Month Calendar Generation

When a financial year is created (e.g., 2026-04-01 to 2027-03-31), `FinancialYearsService` automatically generates 12 sequential monthly accounting periods:

- Month 1: April 2026 (2026-04-01 to 2026-04-30)
- Month 2: May 2026 (2026-05-01 to 2026-05-31)
- ...
- Month 12: March 2027 (2027-03-01 to 2027-03-31)

5.2 Three-Tier Period Lockdown State Machine



1. **`OPEN`:** Standard daily operations. All authorized roles may create and post journals.

2. **`SOFT_LOCKED`:** Month-end review mode. Standard bookkeepers cannot post. Senior accountants and administrators can post with audited override.

3. **`HARD_LOCKED`:** Statutory year-end close. Database triggers and application services reject any post attempts unconditionally (`ERR_PERIOD_HARD_LOCKED`).

6. Contacts & Commercial Subledgers (`src/contacts/`)

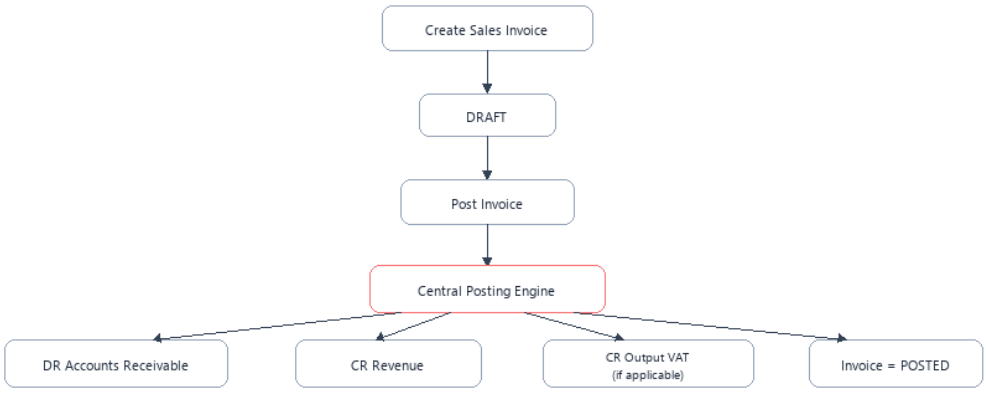


Figure 5 - Sales invoice accounting flow

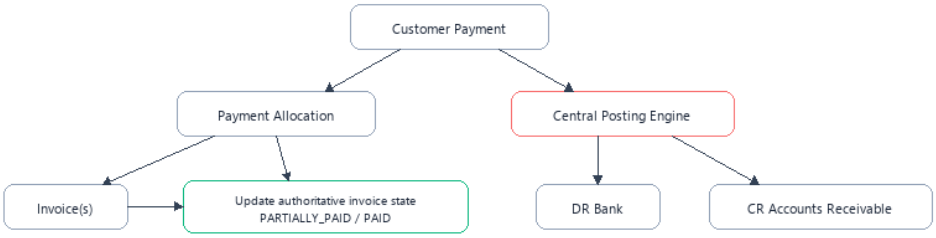


Figure 6 - Customer payment allocation and accounting flow

Figure 6 - Customer payment allocation and accounting flow

6.1 `ContactsService`: Customer/Supplier Directory & Master Data

Manages commercial trading entities:

- Classification: `CUSTOMER`, `SUPPLIER`, or `BOTH`.
- Company registration numbers and VAT identifiers.
- Contact People directory (`ContactPerson`) for invoicing email dispatch.

6.2 `ContactSubledgerService`: Dynamic AR/AP Derivation & Zero Cache Drift

Rather than maintaining a cached `current_balance` column that can drift out of sync with the General Ledger, customer and supplier balances are derived dynamically from posted journal lines:

$$ ext{Customer AR Balance} = \sum(ext{Debits to Accounts Receivable}) - \sum(ext{Credits to Accounts Receivable})$$

```
@Injectable()
export class ContactSubledgerService {
  async getCustomerBalance(organizationId: string, contactId: string): Promise<Decimal> {
    const lines = await this.prisma.journalLine.findMany({
      where: {
        organizationId,
        contactId,
        journalEntry: { status: 'POSTED' },
        account: { accountSubtype: 'ACCOUNTS_RECEIVABLE' },
      },
      select: { debit: true, credit: true },
    });

    let balance = new Decimal(0);
    for (const line of lines) {
      balance = balance.plus(line.debit).minus(line.credit);
    }
    return balance;
  }
}
```

6.3 Customer & Supplier Statement Generation

Generates structured statements for commercial counterparties:

- Opening balance calculation prior to start date.
- Itemized transaction list: Invoices, Credit Notes, and Payments.
- Running balance per transaction.
- Aging analysis breakdown: Current, 30 Days, 60 Days, 90+ Days.

7. Sales Invoicing & Commercial Billing (`src/invoices/`)

7.1 Invoice Creation, Line Pricing, and UK VAT Calculation

```
calculateInvoiceTotals(lines: CreateInvoiceLineDto[]) {
  let subtotal = new Decimal(0);
  let taxTotal = new Decimal(0);

  const calculatedLines = lines.map((l) => {
    const qty = this.money.toDecimal(l.quantity);
```

```
    const price = this.money.toDecimal(l.unitPrice);
    const taxRate = this.money.toDecimal(l.taxRate || 0);

    const lineSubtotal = qty.times(price);
    const lineTax = lineSubtotal.times(taxRate.dividedBy(100));
    const lineTotal = lineSubtotal.plus(lineTax);

    subtotal = subtotal.plus(lineSubtotal);
    taxTotal = taxTotal.plus(lineTax);

    return { ...l, taxAmount: lineTax, lineTotal };
  });

  return {
    subtotal: this.money.round(subtotal, 2),
    taxTotal: this.money.round(taxTotal, 2),
    totalAmount: this.money.round(subtotal.plus(taxTotal), 2),
    calculatedLines,
  };
}
```

7.2 The Automated General Ledger Posting Pipeline (`postInvoice`)

When an invoice is approved and posted, `InvoicesService.postInvoice` atomically generates, validates, posts, and links a balanced double-entry journal entry:

```
async postInvoice(organizationId: string, invoiceId: string, orgContext: OrgContext, actor: Actor) {
  this.postingService.checkPostingPermission(orgContext);

  return this.prisma.transaction(async (tx) => {
    // 1. Lock invoice row FOR UPDATE
    await tx.$queryRaw`
      SELECT id, status FROM invoices
      WHERE id = ${invoiceId}::uuid AND organization_id = ${organizationId}::uuid
      FOR UPDATE
    `;

    const invoice = await tx.invoice.findFirst({
      where: { id: invoiceId, organizationId },
      include: invoiceInclude,
    });

    if (invoice.status !== 'DRAFT') {
      throw new DomainException('INVOICE_INVALID_STATE', 'Only DRAFT invoices can be posted');
    }

    // 2. Identify Accounts Receivable account (nominal 1100)
    const arAccount = await this.resolveArAccount(tx, organizationId, invoice.contact);

    // 3. Identify Tax Payable account (nominal 2100) if tax exists
    const taxAccount = await this.resolveTaxAccount(tx, organizationId, invoice.taxTotal);

    // 4. Construct Journal Lines:
    // Debit AR (1100) with Total Amount
```

```
    const journalLines = [
      {
        organizationId,
        lineNumber: 1,
        accountId: arAccount.id,
        contactId: invoice.contactId,
        debit: invoice.totalAmount,
        credit: new Decimal(0),
        description: `Invoice ${invoice.invoiceNumber} receivable`,
      },
    ];

    let lineNum = 2;
    // Credit Revenue accounts per invoice line
    for (const line of invoice.lines) {
      journalLines.push({
        organizationId,
        lineNumber: lineNum++,
        accountId: line.accountId,
        contactId: invoice.contactId,
        debit: new Decimal(0),
        credit: line.lineTotal.minus(line.taxAmount),
        description: line.description,
      });
    }

    // Credit Tax Payable if tax > 0
    if (this.money.toDecimal(invoice.taxTotal).greaterThan(0)) {
      journalLines.push({
        organizationId,
        lineNumber: lineNum++,
        accountId: taxAccount.id,
        contactId: null,
        debit: new Decimal(0),
        credit: invoice.taxTotal,
        description: `VAT output on invoice ${invoice.invoiceNumber}`,
      });
    }

    // 5. Create Draft Journal Entry
    const journal = await tx.journalEntry.create({
      data: {
        organizationId,
        journalNumber: `TEMP-${invoice.invoiceNumber}`,
        postingDate: invoice.issueDate,
        status: 'DRAFT',
        journalType: 'SALES_INVOICE',
        sourceType: 'INVOICE',
        sourceId: invoice.id,
        narration: `Sales Invoice ${invoice.invoiceNumber}`,
        lines: { create: journalLines },
      },
    });

    // 6. Validate & Post via Accounting Engine
    await this.validator.validateJournalEntry(organizationId, journal.id);
    const postedJournal = await this.postingService.postJournal(organizationId, journal.id, actor);
```

```
    // 7. Update Invoice status to POSTED and link journal ID
    await tx.invoice.update({
      where: { id: invoiceId },
      data: {
        status: 'POSTED',
        journalEntryId: postedJournal.id,
      },
    });

    return postedJournal;
  });
}
```

7.3 Invoice Voiding and Reversal Alignment (`voidInvoice`)

Voiding an invoice that has been posted to the General Ledger executes an atomic reversal:

1. Locks the invoice row `FOR UPDATE`.
2. Locates the linked journal entry (`invoice.journalEntryId`).
3. Invokes `JournalReversalService.reverseJournal`, generating the GAAP counter-journal.
4. Updates the invoice status to `VOIDED`.
5. Emits `AuditEvents.INVOICE_VOIDED` audit log.

8. Financial Reporting Engine (`src/reports/`)

8.1 `TrialBalanceService`: Real-Time Nominal Aggregation

Generates the authoritative Trial Balance report directly from posted journal lines:

- **Integrity Gate:** Filters strictly by `status: { in: ['POSTED', 'REVERSED'] }`. Draft and validated journals are excluded.
- **Balance Computation:**
- For Asset/Expense accounts (`DEBIT` normal): `Net Balance = Gross Debit - Gross Credit`. If positive, placed in Debit column; if negative, placed in Credit column.
- For Liability/Equity/Revenue accounts (`CREDIT` normal): `Net Balance = Gross Credit - Gross Debit`. If positive, placed in Credit column; if negative, placed in Debit column.
- **Equilibrium Check:** Calculates `diff = |totalDebits - totalCredits|`. Sets `isBalanced: diff.isZero()`.
- **Export Capabilities:** Supports both JSON and RFC-4180 compliant CSV formatting.

8.2 `ProfitAndLossService`: Revenue & Expense Classification

Generates the Income Statement over a specified date range:

- **Turnover / Revenue:** Sum of credit balances on nominal accounts 4000–4999.
- **Cost of Sales:** Sum of debit balances on nominal accounts 5000–5999.
- **Gross Profit:** $ ext{Gross Profit} = ext{Revenue} - ext{Cost of Sales}$.
- **Operating Expenses (OPEX):** Sum of debit balances on nominal accounts 6000–8999 (Rent, Salaries, Software).
- **Operating Profit:** $ ext{Operating Profit} = ext{Gross Profit} - ext{Operating Expenses}$.
- **Net Profit / (Loss):** Final net income transferred to Retained Earnings in the Balance Sheet.

8.3 `BalanceSheetService`: Assets, Liabilities, and Equity Verification

Verifies the foundational accounting equation at a specific date:

$$ ext{Assets} = ext{Liabilities} + ext{Equity}$$

- **Current & Non-Current Assets:** Cash at Bank (1000), Accounts Receivable (1100), Equipment (1500).
- **Current & Non-Current Liabilities:** Accounts Payable (2000), VAT Payable (2100), Bank Loans (2500).
- **Equity:** Share Capital (3000) + Prior Year Retained Earnings (3100) + Current Year Net Profit (derived from P&L).

9. Security, Authentication & Multi-Tenancy (`src/auth/`, `src/roles/`, `src/organizations/`, `src/audit/`)

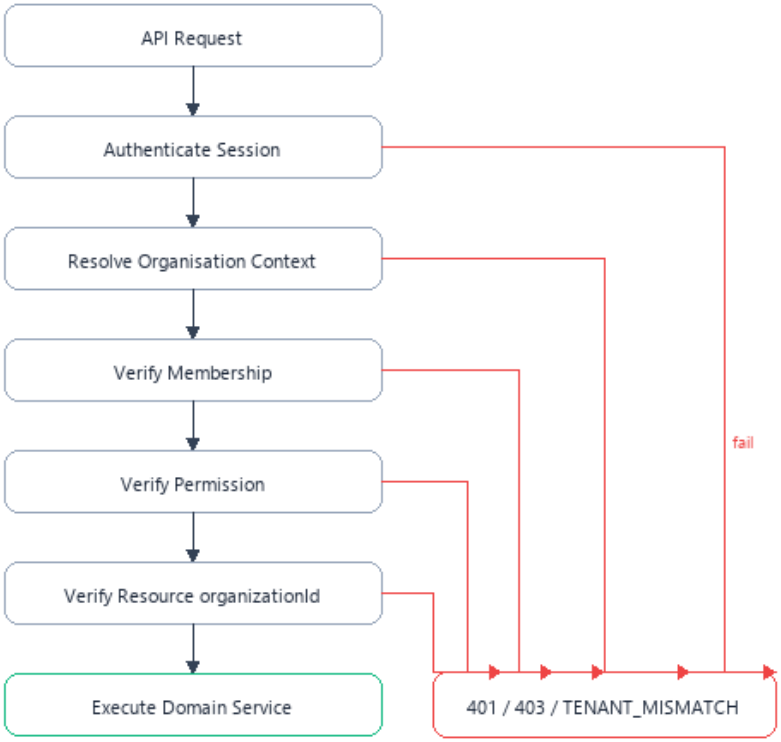


Figure 7 - Request authorisation and tenant-isolation flow

Figure 7 - Request authorisation and tenant-isolation flow

9.1 `PasswordService`: scrypt Hashing & Constant-Time Verification

```
@Injectable()
export class PasswordService {
  async hash(password: string): Promise<string> {
    const salt = crypto.randomBytes(16).toString('hex');
    const derivedKey = (await promisify(crypto.scrypt)(password, salt, 64)) as Buffer;
    return `${salt}:${derivedKey.toString('hex')}`;
  }

  async verify(password: string, storedHash: string): Promise<boolean> {
    const [salt, key] = storedHash.split(':');
    const keyBuffer = Buffer.from(key, 'hex');
    const derivedKey = (await promisify(crypto.scrypt)(password, salt, 64)) as Buffer;
    return crypto.timingSafeEqual(keyBuffer, derivedKey);
  }
}
```

- **Cryptographic Guarantees:** 16-byte cryptographically secure random salt, 64-byte key length. `crypto.timingSafeEqual` prevents side-channel timing attacks during password verification.

9.2 `AuthGuard`: Bearer Token Extraction & Session Validation

```
@Injectable()
export class AuthGuard implements CanActivate {
  async canActivate(context: ExecutionContext): Promise<boolean> {
    const req = context.switchToHttp().getRequest();
    const authHeader = req.headers['authorization'];
    if (!authHeader || !authHeader.startsWith('Bearer ')) {
      throw new UnauthorizedException('Authentication required');
    }

    const token = authHeader.substring(7);
    const session = await this.prisma.userSession.findUnique({
      where: { token },
      include: { user: true },
    });

    if (!session || session.expiresAt < new Date()) {
      throw new UnauthorizedException('Session expired or invalid');
    }

    req.user = session.user;
    return true;
  }
}
```

9.3 `RolesGuard`: Role-Based Access Control

Evaluates required role permissions declared via `@Roles(...)`:

- **Role Hierarchy:**
- `OWNER`: Full tenant permissions, billing, organization deletion.
- `ADMINISTRATOR`: User management, financial configuration, periods.
- `ACCOUNTANT`: Journal creation, posting, period locks, reporting.
- `BOOKKEEPER`: Draft invoices, draft journals, customer records.
- `VIEWER`: Read-only reporting access.

9.4 `AuditService`: Structured Immutable Logging

Every mutating action generates an audit entry capturing actor, organization, entity ID, and state deltas (`oldValues` and `newValues`):

- Backed by PostgreSQL `trg_audit_logs_immutable` trigger preventing any update or deletion.

10. AI Document Intelligence & Safety Guardrails (`src/ai/`)

10.1 `AccountingClassificationService`: Document Code Mapping

Uses heuristic classification and LLM embeddings to suggest nominal accounts for inbound supplier bills and receipts:

- Extracts merchant name, line descriptions, VAT amounts.
- Maps recurring suppliers to standard OPEX accounts (e.g. AWS hosting -> 8000 Software & Hosting).

10.2 `TransactionDirectionService`: Debit/Credit Prediction

Determines transaction direction based on document semantics:

- Sales Invoices: Debit Accounts Receivable (1100), Credit Revenue (4000), Credit VAT Output (2100).
- Supplier Bills: Debit Expense (e.g. 8000), Debit VAT Input (2100), Credit Accounts Payable (2000).

10.3 `AccountingSafetyService`: Validation Guardrails

Prevents automated AI actions from compromising financial integrity:

- Prohibits autonomous posting to control accounts.
- Enforces confidence thresholds: predictions below 85% confidence are flagged for human review.
- Rejects any AI suggestion that violates double-entry balance constraints.

11. Common Infrastructure, Filters, Pipes & Logger (`src/common/`)

11.1 `AllExceptionsFilter`: Centralized Domain Error Envelope

Catches all thrown exceptions and formats them into a predictable, RFC-7807 compatible JSON structure:

```
{
  "success": false,
  "error": {
    "code": "ERR_JOURNAL_UNBALANCED",
    "message": "Debits (100.00) must equal credits (90.00)",
    "timestamp": "2026-10-12T00:00:00.000Z",
    "requestId": "9b1deb4d-3b7d-4bad-9bdd-2b0d7b3dcb6d"
```

```
  }
}
```

11.2 `DomainException`: Standardized Error Structure

Enables strong error categorization across services with domain-specific error codes (e.g., `ERR_PERIOD_HARD_LOCKED`, `ERR_CONTROL_ACCOUNT_DIRECT_POSTING`).

11.3 `ValidationPipe`: Class-Validator Sanitization

Configured with `whitelist: true`, `forbidNonWhitelisted: true`, and `transform: true` to prevent parameter injection attacks.

11.4 `AppLogger`: Correlation IDs and Structured JSON Logging

Attaches `x-request-id` to every log line, producing structured JSON outputs ingested by AWS CloudWatch.

12. Web Client Architecture (`apps/web/src/`)

12.1 App Router Layout & Authentication Context

Built on **Next.js 15** with React 19 and Turbopack:

- App router structure (`apps/web/src/app/`).
- Persistent authentication context (`auth-context.tsx`) managing session storage, active organisation state, and token refresh.

12.2 Tenant-Scoped API Client (`api.ts`)

The frontend API wrapper (`apps/web/src/lib/api.ts`) automatically resolves and prefixes tenant paths:

```
export function resolveEndpoint(endpoint: string, activeOrgId?: string | null): string {
  let clean = endpoint.startsWith('/') ? endpoint : `/${endpoint}`;

  if (activeOrgId && !clean.startsWith('/organizations') && !clean.startsWith('/auth')) {
    if (clean.startsWith('/accounts') || clean.startsWith('/journals') ||
clean.startsWith('/invoices')) {
      clean = `/organizations/${activeOrgId}${clean}`;
    }
  }
  return clean;
}
```

12.3 Key User Interfaces & Components

- **Dashboard:** Displays real-time KPI cards for Cash at Bank, Accounts Receivable, Accounts Payable, and YTD Net Profit.
- **Journal Entry Grid:** Interactive multi-line journal editor with a sticky balance tally bar displaying live Debits, Credits, and Balance Difference in real-time.
- **Invoice Editor & GL Post Button:** Comprehensive invoice builder with automatic tax calculation and one-click General Ledger posting.
- **Customer & Supplier Statements:** Date-filterable statement viewer with instant PDF generation and email dispatch.
- **Financial Reports Viewers:** Interactive Trial Balance, Profit & Loss, and Balance Sheet viewers with CSV export.

13. Cloud Deployment & CI/CD Pipeline

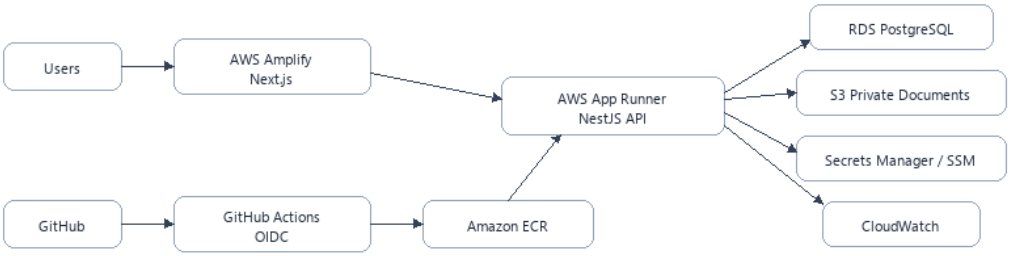


Figure 8 - Low-cost AWS DEV architecture in eu-west-2

Figure 8 - Low-cost AWS DEV architecture in eu-west-2

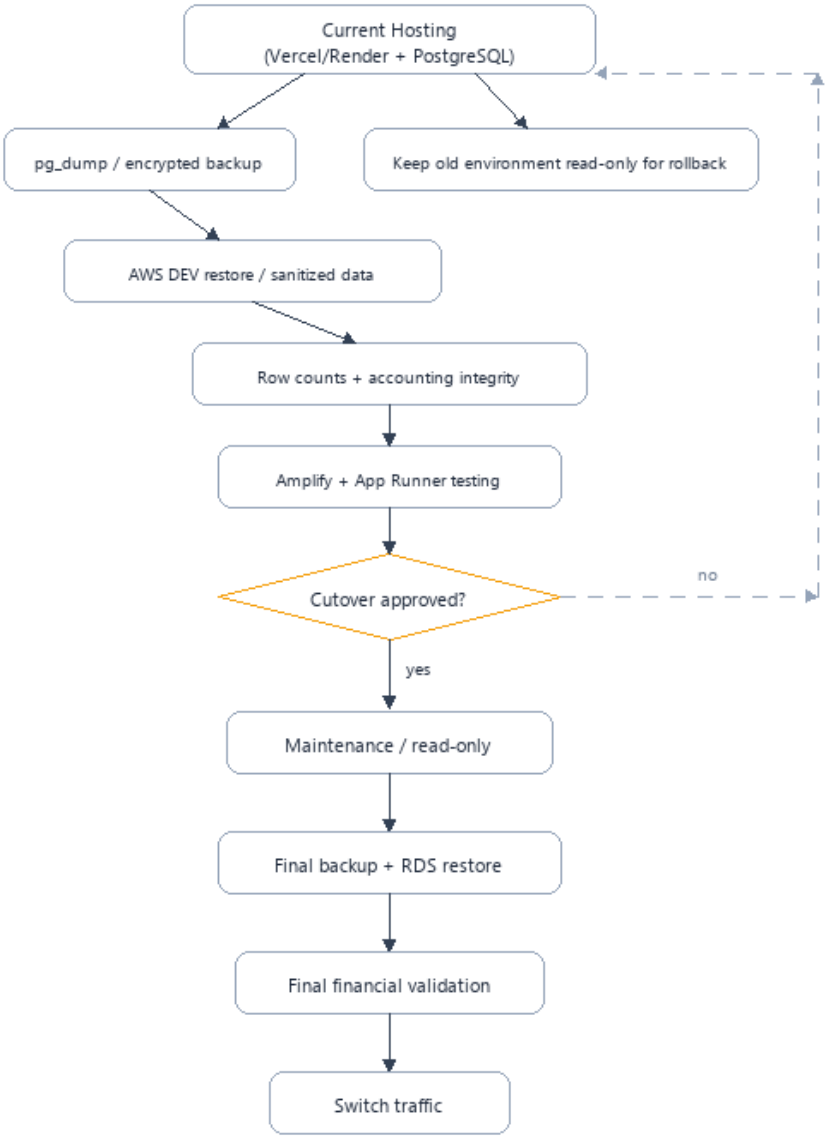


Figure 9 - Safe current-hosting to AWS migration and rollback path

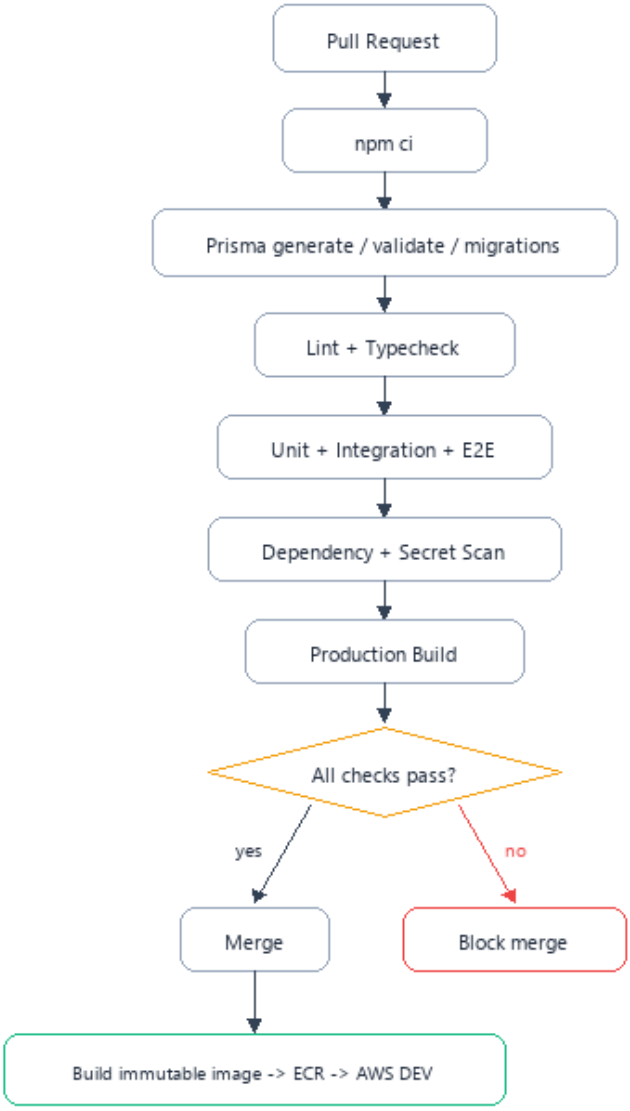


Figure 10 - Pull-request quality gate and AWS DEV deployment flow

13.1 AWS Dev Architecture (`eu-west-2` London)

- **Frontend:** AWS Amplify hosting Next.js 15 SSR build with automated edge caching.
- **Backend:** AWS App Runner running NestJS 11 containerized application with automatic horizontal scaling.
- **Database:** Amazon RDS PostgreSQL 16 Multi-AZ instance in private VPC subnets.

13.2 CI/CD Quality Gate

Every pull request triggers the automated GitHub Actions pipeline:

1. `npm ci`: Clean install across monorepo workspace.
2. `prisma validate`: Schema integrity and relation verification.
3. ESLint & TypeScript compilation checks.
4. **Automated Unit & Service Tests:** 26 test suites / 127 tests must PASS before merge approval.
5. Production container image build and push to Amazon ECR.

END OF BLYNT COMPLETE SOURCE CODE EXPLAINER